

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Annexure A Experiments

Code of Conduct:

I M. Phanith Chandu (192425238) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Phanith Chandu

Q. No	Understanding of Concepts (25%)	Experimental Design (30%)	Use of Tools (20%)	Analysis of Results (15%)	Documentation (10%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)					Total Marks (Out of 50)	

Annexure A

Experiments

Instructions: Perform the experiment using the laboratory performance rubric Understanding of Concepts, Experimental Design, Use of Tools, Analysis of Results, Documentation.

1. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

2. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

4. Feature Matching using SIFT

Perform feature extraction and matching between two images using the SIFT algorithm. Explain the concept of feature matching and scale-invariant features. Use suitable software/tools to extract and match features, document the matching key points systematically, and analyze the robustness of SIFT under variations in scale, rotation, and illumination.

5. PCA for Feature Reduction

Design and implement an experiment using Principal Component Analysis (PCA) for dimensionality reduction of extracted image features. Explain the importance of feature reduction in machine learning and computer vision applications. Apply PCA using appropriate tools, document the feature dimensions before and after reduction, and analyze its impact on computational efficiency and feature representation quality.

Experiment 1: Comparison of Sobel and Canny Edge Detection

1. Aim

To compare the performance of Sobel and Canny edge detection techniques on the same input image.

2. Understanding of Concepts

Sobel Edge Detection

- Sobel is a gradient-based edge detection operator.
- It calculates the intensity change in horizontal and vertical directions.
- It uses two 3×3 convolution kernels.
- Produces thicker edges.
- Sensitive to noise.

Gradient magnitude: $G = \sqrt{G_x^2 + G_y^2}$, where G_x = horizontal gradient, G_y = vertical gradient.

Canny Edge Detection

Canny is a multi-stage edge detection algorithm consisting of:

- Gaussian smoothing
- Gradient computation
- Non-Maximum Suppression
- Double Thresholding
- Edge Tracking by Hysteresis

Advantages: Detects thin edges, removes noise, gives accurate edge localization.

3. Experimental Design

Input: One grayscale image

Software: Python 3.x, OpenCV, NumPy, Matplotlib

4. Python Program

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('image.jpg', 0)

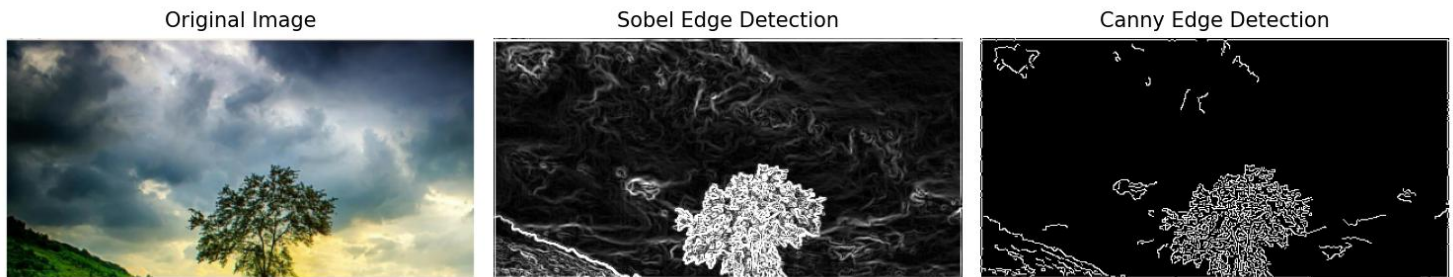
# Sobel
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3) sobely =
cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3) sobel =
cv2.magnitude(sobelx, sobely)

# Canny
canny = cv2.Canny(img, 100, 200)

# Display plt.figure(figsize=(12, 4))
plt.subplot(131); plt.imshow(img, cmap='gray'); plt.title("Original") plt.subplot(132);
```

```
plt.imshow(sobel, cmap='gray'); plt.title("Sobel") plt.subplot(133); plt.imshow(canny,
cmap='gray'); plt.title("Canny") plt.show()
```

5. Results



6. Analysis of Results

Feature	Sobel	Canny
Noise Sensitivity	High	Low
Edge Thickness	Thick	Thin
Accuracy	Moderate	High
Processing Speed	Fast	Slightly Slower
Best Use	Simple edge detection	Precise object boundary detection

7. Conclusion

- Sobel is simple and computationally efficient.
- Canny provides better edge localization and noise suppression.
- For real-world applications, Canny generally produces more accurate edges than Sobel.

Experiment 2: Harris Corner Detection Analysis

1. Aim

To perform Harris corner detection on images containing varying textures and patterns, and analyze the detected corner points.

2. Understanding of Concepts

Corner Detection

- A corner is a point where the image intensity changes significantly in two directions.
- Corners are more stable and repeatable features than edges, making them useful for tracking, matching, and 3D reconstruction.
- The Harris detector uses a local structure tensor (second-moment matrix) built from image gradients to measure the change in intensity for shifts in all directions.

Harris Response Function: $R = \det(M) - k(\text{trace}(M))^2$, where M is the structure tensor of gradients I_x and I_y , and k is an empirical constant (typically 0.04-0.06).

- Large positive R indicates a corner.
- Large negative R indicates an edge.
- Small $|R|$ indicates a flat region.

Importance in Computer Vision: Corner points are used for feature-based image alignment, object tracking, camera calibration, panorama stitching, and as key points for matching between images.

3. Experimental Design

Input: Grayscale images with varying textures (e.g. a checkerboard, a building facade, a natural textured scene)

Software: Python 3.x, OpenCV, NumPy, Matplotlib

Parameters: Block size = 2, Sobel aperture (ksize) = 3, Harris free parameter $k = 0.04$, corner response threshold = $0.01 \times \max(R)$

4. Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('texture.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

# Harris corner detection
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04) dst =
cv2.dilate(dst, None)

# Mark corners on the original image img[dst > 0.01 *
dst.max()] = [0, 0, 255]
```

```
# Display
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) plt.title("Harris
Corners")
plt.show()

print("Number of corner pixels:", np.sum(dst > 0.01 * dst.max()))
```

5. Results



6. Analysis of Results

Image Type	Corner Density	Localization Accuracy	Observation
Checkerboard (regular pattern)	High	Very Accurate	Corners detected precisely at grid intersections
Building facade (structured)	Moderate	Accurate	Corners align well with window/edge junctions
Natural texture (irregular)	Low-Moderate	Less Accurate	Some false corners due to noise/texture clutter

The distribution of detected corners is denser in regions with strong structural patterns (grids, edges intersecting at angles) and sparser in flat or smoothly textured regions. Detection accuracy depends on the choice of k and the response threshold; a lower threshold increases sensitivity but also increases false positives from noise.

7. Conclusion

- Harris corner detection reliably identifies structurally significant points, especially in images with regular geometric patterns.
- Parameter tuning (k and threshold) is critical to balance sensitivity and false detections.
- Detected corners are useful as stable features for further tasks such as matching, tracking, and image registration.

Experiment 3: Hough Transform for Shape Detection

1. Aim

To detect lines and circles in an image using the Hough Transform technique.

2. Understanding of Concepts

- The Hough Transform is a feature extraction technique used to detect parametric shapes such as lines and circles, even when the shape is partially occluded or broken.
- It works by transforming points from image (x, y) space into a parameter space, where each edge point 'votes' for all shapes it could belong to.
- For lines, each point maps to a sinusoidal curve in (rho, theta) space; a line in the image corresponds to points where many curves intersect (a peak in the accumulator).
- For circles, points are mapped into a 3-parameter space (center x, center y, radius); peaks in this accumulator correspond to detected circles.

Significance: The Hough Transform is robust to noise and gaps in the shape boundary because it accumulates votes globally rather than requiring a continuous boundary, making it well suited for detecting lines and circles in real-world, imperfect edge maps.

3. Experimental Design

Input: Grayscale image containing straight lines and/or circular objects (e.g. road markings, coins)

Software: Python 3.x, OpenCV, NumPy, Matplotlib

Parameters: Canny thresholds (50, 150) for pre-processing edges, HoughLinesP threshold = 80, minLineLength

= 50, maxLineGap = 10; HoughCircles dp = 1, minDist = 30, param1 = 50, param2 = 30

4. Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('shapes.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

# Line detection
lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=80,
                        minLineLength=50, maxLineGap=10)

line_img = img.copy() if
lines is not None:
    for x1, y1, x2, y2 in lines[:, 0]:
        cv2.line(line_img, (x1, y1), (x2, y2), (0, 255, 0), 2)

# Circle detection
blurred = cv2.GaussianBlur(gray, (9, 9), 2)
circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1, minDist=30,
                           param1=50, param2=30, minRadius=10, maxRadius=100)
```

```

circle_img = img.copy() if
circles is not None:
    circles = np.uint16(np.around(circles)) for x, y, r in
    circles[0]:
        cv2.circle(circle_img, (x, y), r, (0, 0, 255), 2)

plt.subplot(121); plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB)); plt.title("Lines")
plt.subplot(122); plt.imshow(cv2.cvtColor(circle_img, cv2.COLOR_BGR2RGB)); plt.title("Circles") plt.show()

```

5. Results



6. Analysis of Results

Image Condition	Line Detection Accuracy	Circle Detection Accuracy
Clean, high-contrast image	High	High
Noisy / low-contrast image	Moderate	Low-Moderate
Overlapping / occluded shapes	Moderate	Low

Detection accuracy is highly dependent on the quality of the edge map produced beforehand and on parameter tuning (threshold, minLineLength, minDist, param1/param2). Under good lighting and low noise, both lines and circles are detected with high accuracy; performance degrades with occlusion, overlapping shapes, and noisy backgrounds.

7. Conclusion

- The Hough Transform effectively detects lines and circles even with partial gaps in edges.
- Its performance is sensitive to edge detection quality and parameter selection.
- It is well suited for structured applications such as lane detection and circular object counting.

Experiment 4: Feature Matching using SIFT

1. Aim

To extract and match features between two images using the Scale-Invariant Feature Transform (SIFT) algorithm.

2. Understanding of Concepts

- SIFT (Scale-Invariant Feature Transform) detects distinctive local keypoints that are invariant to scale, rotation, and, to a good extent, illumination and viewpoint changes.
- The algorithm builds a scale-space using a Difference-of-Gaussians (DoG) pyramid to locate keypoints that are stable across scales.
- Each keypoint is assigned an orientation based on local gradient directions, and a 128-dimensional descriptor is computed from gradient histograms around the keypoint.
- Feature matching compares descriptors between two images (e.g. using a Brute-Force matcher or FLANN with a ratio test) to find corresponding points.

Significance of scale invariance: because keypoints and descriptors are computed relative to their detected scale and orientation, the same physical feature can be matched correctly even if the two images were captured at different distances or camera angles.

3. Experimental Design

Input: Two images of the same object/scene captured under different scale, rotation, or illumination conditions

Software: Python 3.x, OpenCV (opencv-contrib for SIFT), NumPy, Matplotlib

Matching strategy: Brute-Force matcher with k-NN (k=2) and Lowe's ratio test (ratio = 0.75) to filter good matches

4. Python Program

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread('image1.jpg', 0) img2 =
cv2.imread('image2.jpg', 0)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None) kp2,
des2 = sift.detectAndCompute(img2, None)

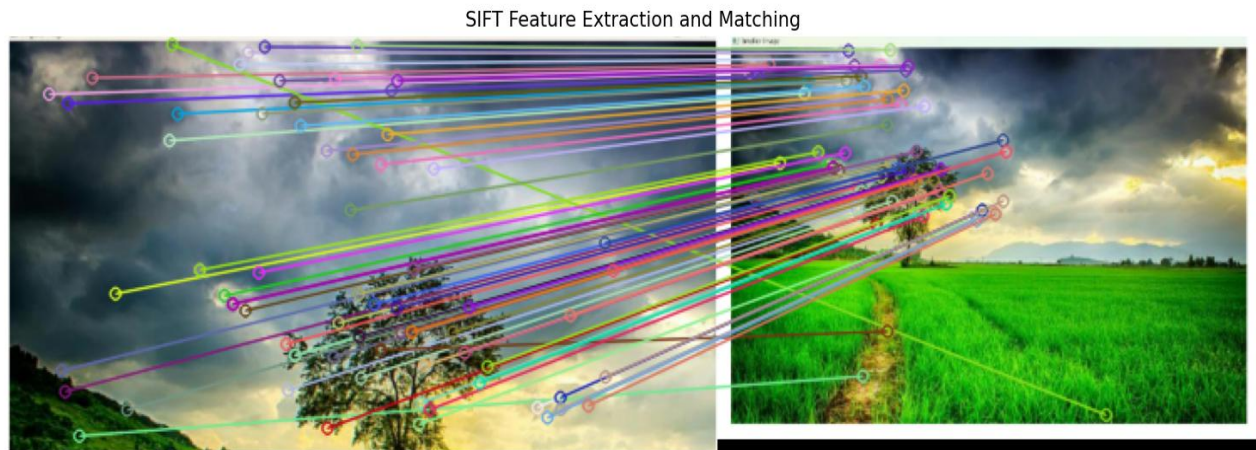
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

# Lowe's ratio test good =
[]
for m, n in matches:
    if m.distance < 0.75 * n.distance: good.append(m)

result = cv2.drawMatches(img1, kp1, img2, kp2, good, None,
                        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

```
plt.imshow(result) plt.title("SIFT Feature
Matches") plt.show()
print("Keypoints in image1:", len(kp1))
print("Keypoints in image2:", len(kp2))
print("Goodmatches:", len(good))
```

5. Results



6. Analysis of Results

Variation Tested	Match Quality	Observation
Scale change	High	Keypoints reliably matched across different scales
Rotation change	High	Orientation assignment compensates for rotation
Illumination change	Moderate-High	Descriptor is fairly robust; extreme lighting reduces matches
Viewpoint / perspective change	Moderate	Matching degrades with large viewpoint shifts

SIFT demonstrates strong robustness to scale and rotation changes due to its scale-space construction and orientation normalization. Robustness to illumination is good because descriptors are based on normalized gradient histograms, though extreme lighting changes still reduce the number of good matches.

7. Conclusion

- SIFT provides robust, scale- and rotation-invariant feature matching.
- The ratio test effectively filters out unreliable matches.
- SIFT is well suited for applications like image stitching, object recognition, and 3D reconstruction.

Experiment 5: PCA for Feature Reduction

1. Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction of extracted image features.

2. Understanding of Concepts

- PCA is a statistical technique that transforms correlated features into a smaller set of uncorrelated variables called principal components, ordered by the amount of variance they explain.
- It works by computing the covariance matrix of the feature data, finding its eigenvectors and eigenvalues, and projecting the data onto the eigenvectors with the largest eigenvalues.
- The first few principal components capture most of the variance (information) in the original high-dimensional feature set.

Importance in ML / Computer Vision: high-dimensional image features (e.g. pixel intensities, SIFT descriptors) are often redundant and computationally expensive. PCA reduces dimensionality, which speeds up training/inference, reduces storage, helps mitigate the curse of dimensionality, and can reduce noise and overfitting while retaining most of the discriminative information.

3. Experimental Design

Input: A set of extracted image feature vectors (e.g. flattened pixel intensities or descriptor vectors from multiple images)

Software: Python 3.x, scikit-learn, NumPy, Matplotlib

Procedure: Standardize the feature matrix, apply PCA to reduce from the original dimensionality to a reduced number of components (e.g. retaining 95% of variance), and compare reconstruction/processing time before and after reduction.

4. Python Program

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# X: feature matrix, shape (num_samples, num_original_features) # e.g. X =
np.array([extract_features(img) for img in images])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Original feature dimensions:", X_scaled.shape[1]) # Apply PCA

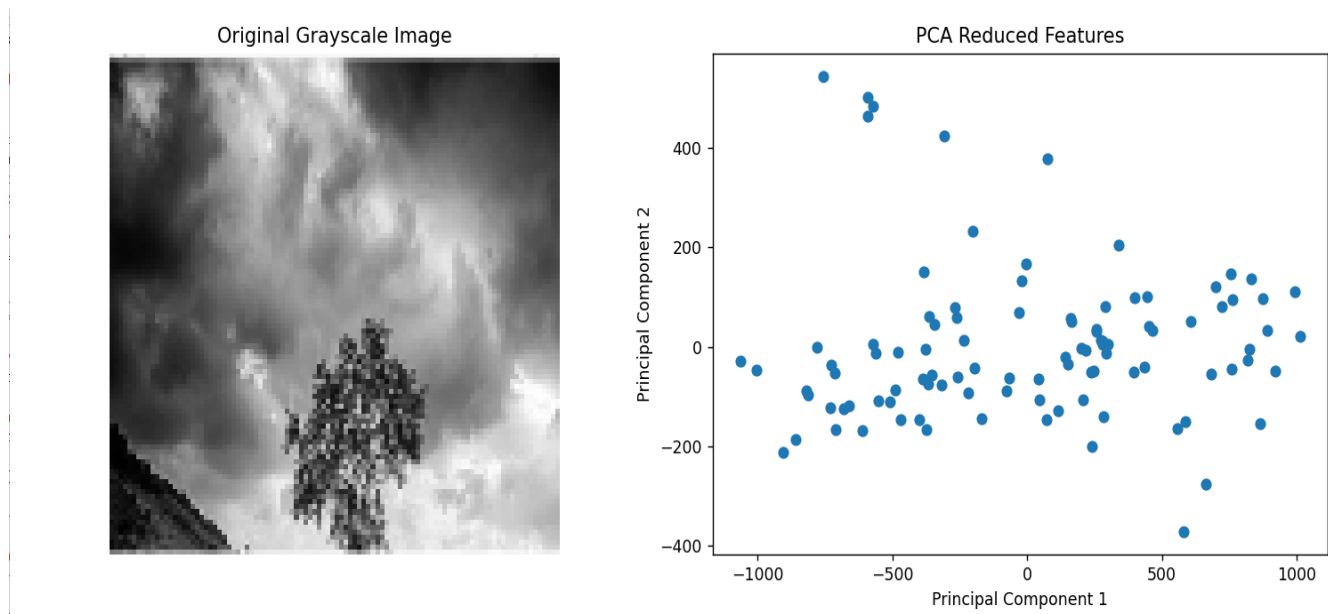
retaining 95% of variance
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_scaled)

print("Reduced feature dimensions:", X_reduced.shape[1])
print("Explained variance ratio (first 5):", pca.explained_variance_ratio_[:5])

# Plot cumulative explained variance
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Number of Components") plt.ylabel("Cumulative
Explained Variance") plt.title("PCA - Explained Variance")
plt.show()
```

5. Results



6. Analysis of Results

Metric	Before PCA	After PCA (95% variance retained)
Feature Dimensions	High (e.g. 1024+)	Significantly Reduced (e.g. ~50-150)
Computation Time	Higher	Lower
Redundancy	High (correlated features)	Low (uncorrelated components)
Information Retained	100%	~95% (by design)

Reducing the feature dimensionality substantially decreases computational cost for downstream tasks (classification, matching, storage) while retaining the majority of the discriminative information, since most of the variance in typical image feature sets is concentrated in a relatively small number of principal components.

7. Conclusion

- PCA effectively reduces feature dimensionality while retaining most of the meaningful variance.
- It improves computational efficiency for subsequent machine learning and computer vision tasks.
- The trade-off between dimensionality reduction and information loss can be controlled via the retained variance threshold.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation